CO03: The analysis of strange particles

Zichi Zhang
sjoh6079

Feb 2026

Abstract

An analysis of the provided data file has been carried out to obtain the masses and average lifetimes of strange mesons and baryons. The locations of peaks in mass distributions of the strange particles were found to be around 0.50 ± 0.01 GeV/c^2 and 1.115 ± 0.003 GeV/c^2 for K_0^S and Λ^0 particles respectively. Those results are in agreement with the known results of $0.493667 \pm 1.3 \times 10^{-5}$ GeV/c^2 [2] and $1.115683 \pm 6 \times 10^{-6}$ GeV/c^2 [3]. However, the lifetime estimate of the strange mesons ($ct = 7.29 \pm 5.14cm$) illustrates a high uncertainty that is likely caused by systematic errors.

1 Introduction

Because of symmetry, the existence of strange particles was predicted before any experimental discovery. Starting from 1985, Fermilab has started to conduct experiments of collisions between high energy particles. However, the resultant data file contains large amounts of events (around 10^4), so data analysis was vital for extracting the relevant information. By reconstructing the data analysis process,a modular programming workflow that divides tasks into smaller subtasks could be developed. In this report, section 2 outlines the purpose of core classes and justifies the mathematical and physical aspects of the methods used. Section 3 discusses the major results and errors, while section 4 provides a short analysis of performance optimization. Section 5 concludes the report. Detailed explanations of the methods used in scripts will not be included because they are included as comments of scripts.

2 Methodology and Theory

While explanation of methods in other classes are mostly given by comments in individual scripts, the difficulty sometimes also arises from lack of transparency of tasks and factors independent of programming perspective, such as mathematics and physics. This section therefore provides explanations to those issues, started with the division of tasks discussed in subsection 2.1. Then, in subsection 2.2, diagrams will be drawn to illustrate essential geometry used in the scripts, followed by the physical arguments for the numerical values chosen for cuts on likely candidates in subsection 2.3.

2.1 Dividing tasks between classes

In modular programming practice, it is important to specify the role each class plays. The classes used are roughly divided into the groups outlined below:

- handles different aspects of raw data: `CdfDataFile.m`,`CdfTrack.m`,`CdfEvent.m`
- provides mathematical treatments on loaded data: `vertex.m`,`helix.m`
- actual analysis: `Analysis.m`, `KOSAnalysis.m`, `LambdaAnalysis.m`, `Loop.m`

A brief description of some scripts can be found in [1], and a more detailed explanation is given in the comments of each script.

Apart from CdfTrack, all other classes belong to handle class, which enables a consistent reflection of changes made to properties of an object. CdfTrack was chosen to be a value class as the manual adjustment of track parameters is not required.

2.2 Mathematical convention and explanation

Sign convention: the sign of (half)curvature is defined positive when the helix is formed by a positive charge, then given the direction of external magnetic field, the positive charge is chosen to move anticlockwise. The sign of radius and impact parameter are defined to be the same as that of (half)curvature.

Lengths are measured centimeters, while other variables are expressed in natural units. For example, the charge number of an electron would be -1 in these units.

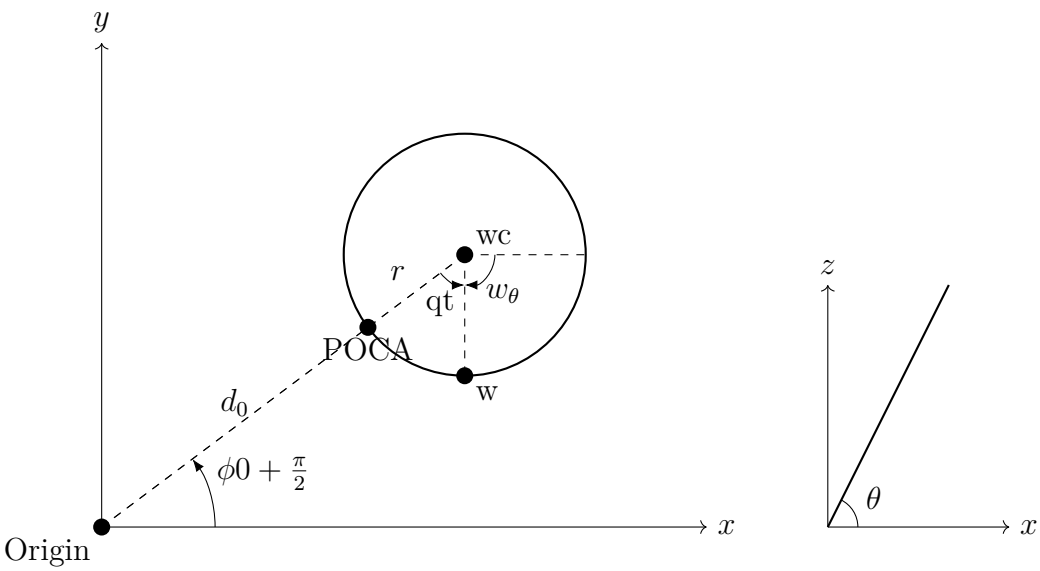

Figure 1: Geometric illustration of upper and front views of a helix

In the track parameters, ϕ_0 is defined as the (signed) angle made by the transverse momentum with the x-axis. In the example illustrated, we have $|\beta| + |\phi 0| = \frac{\pi}{2}$, which is consistent with 1. When particles are traveling anticlockwise, $\pi - |\phi 0| + \beta = \frac{\pi}{2}$ is obtained. Although the angle dependence introduces a negative sign, this effect is balanced by negative radius and impact parameter. 1 is therefore maintained, being indepedent with charge number.

One complexity in obtaining the expression for w is that the change in angles are also signed. Since $q = \pm 1$ for positive and minus charge respectively, the charge number is consistent with the sign convention of rotation. For example, when $q = -1$, the particle will move clockwise, which corresponds to a negative signed angle. Therefore, an extra factor of q is added to t to account for the dependence on orientation of rotation. Using $-w_\theta + qt + (\phi 0 + \frac{\pi}{2}) = \pi$, one approaches 2.

The z representation contains the constant term and the vertical shift. In the expression of z, $|r|t$ is the projection of displacement in xy plane, while θ is the inclination angle made by the helix with xy plane. The expression of z(t) is therefore 3.

$$w_c = (r + d_0)e^{i\left(\phi_0 + \frac{\pi}{2}\right)} \tag{1}$$

$$w(t) = w_c - ire^{i(\phi_0 + qt)} \tag{2}$$

$$z(t) = z_0 + |r|\, t \cot\theta \tag{3}$$

(t is not time in these formula, but the angle labeled in the diagram) After writting out the expressions related to a single helix, it is also useful to be able to compute intersections of helices.

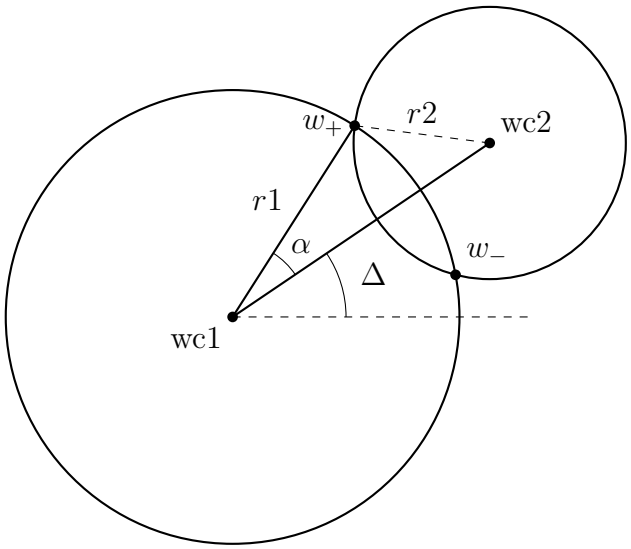

Figure 2: illustration of two helices intersecting

By using trigonometric relationships, it is then straightforward to obtain equations :

$$\cos\alpha = \frac{|w_{c2} - w_{c1}|^2 + r_1^2 - r_2^2}{2\,|w_{c2} - w_{c1}|\;|r_1|} \tag{4}$$

$$\tan\Delta = \frac{y_{c2} - y_{c1}}{x_{c2} - x_{c1}} \tag{5}$$

$$w_\pm = w_{c1} + |r_1|\, e^{i(\Delta \pm \alpha)} \tag{6}$$

The expressions 6 can then be substituted back to 2,producing the t arguments to the intersections, allowing the complete set of coordinates of intersections to be found.

2.3 Physical justifications for the cuts applied

2.3.1 in K0SAnalysis.m

In order to effectively select strange kaons, a number of cuts on parameters are applied to filter particles which are unlikely to be K_0^S. The physical considerations are listed below:

- The known average lifetime of strange kaons is ct=2.6844cm [1], it is then argued that the horizontal flight distance should exceed a minmum value, which was chosen to be 2cm. However, if the distribution of flight distance from primary vertex is exponential, more than half of the actual strange mesons will also be filtered out (Appendix B.1). This concern will be postponed until section 3.2.2.

- Since "Beams of protons and antiprotons collide at the center of CDF at nearly the speed of light" [4], the strange kaons which can only be produced by strong interactions should not be found far from origin. Therefore, those candidate particles with impact parameter greater than $0.5cm$ are filtered out.

- Another consequence of formation of strange kaons via high-energy collision near light speed is the large resultant energy, thus a cut on the minimum transverse momentum of individual tracks is set at $1Gev/c$.

- Given the horizontal flight distance and momentum, the true impact parameters of daughter tracks should not be small. This is because the filter demands the decay to take place at at least 2 centimeters away from primary vertex.

2.3.2 in LambdaAnalysis.m

Most of the cuts applied in LambdaAnalysis are identical. The reasons behind additional or different cuts are listed below:

- higher momentum filter on one of the daughters. With similiar concerns, only tracks with relatively high transverse momentum are likely to produce Λ^0. The difference is that since most of the horizontal momentum will be in distributed in the more energetic proton [1], likely daughters are required to exceed a higher value, at $2Gev/c^2$.

- less true impact parameter of daughters compared to K_0^S. Assume the scale of energy for producing Λ^0 is similiar than that of K_0^S. Since $E^2 = m^2 + p^2$(in natural units), the more massive Λ^0 is expected to have a smaller momentum, leading to less impact parameters of its daughters.

3 Results and Analysis

3.1 masses of K_0^S and Λ^0

After applying the filters for raw mass data of strange mesons and baryons, the filted distribution can then be plotted into mass histograms. The resultant distributions are displayed in the next page:

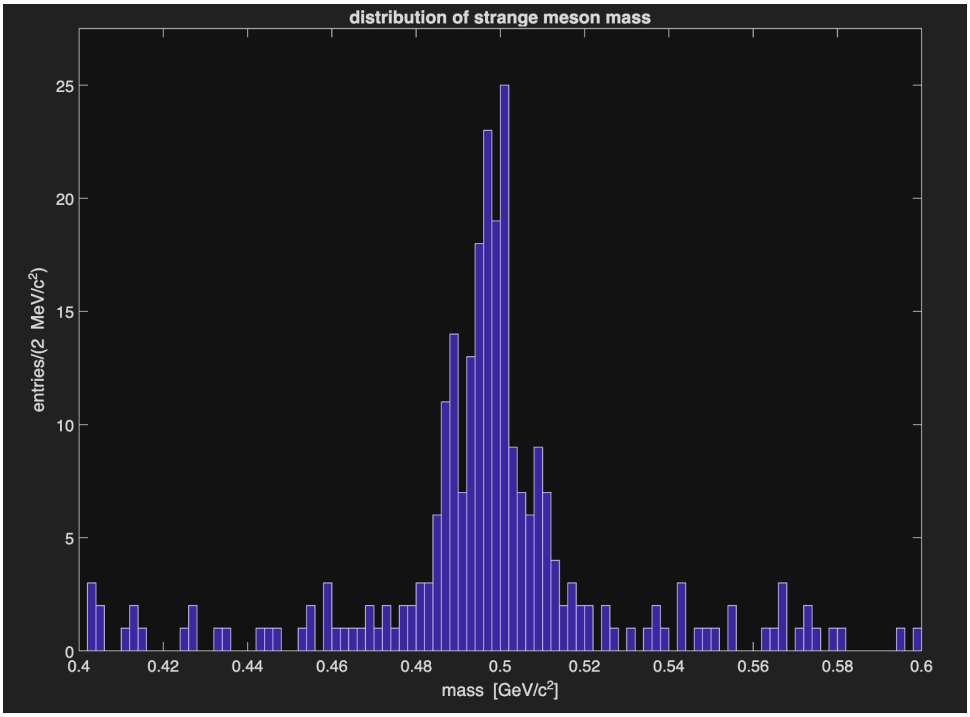

Figure 3: mass distribution of selected particles which are likely K_S^0. The horizontal axis of the histogram is separated into 100bins, with lower limit at $0.4Gev/c^2$ and upper limit at $0.6Gev/c^2$

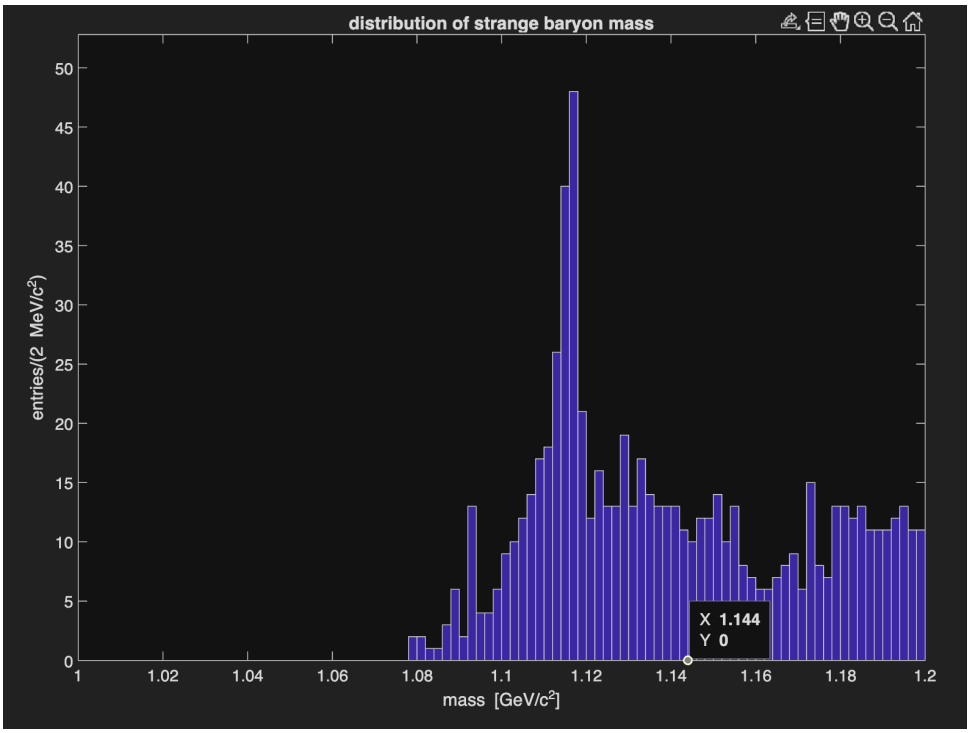

Figure 4: mass distribution of selected particles which are likely Λ^0. The histogram is separated into 100bins, with lower limit at $1Gev/c^2$ and upper limit at $1.2Gev/c^2$

As can be seen in 3, the distribution pattern is nearly symmetric about the peak, which is at $0.5Gev/c^2$ with a width of about $4Mev/c^2$. This finite width is due to the

resolution of histogram, whose bin width is $2Mev/c^2$.(B.3) As a sign of reasonable cut, the noise far from the peak is negligible. Therefore, by assuming an approximate Gaussian distribution, and the error was found to be $0.01Gev/c^2$.

On the other hand, the mass distribution of strange baryons in 4 has a sharp cutoff at around $1.07Gev/c^2$. This feature is expected, since in the rest frame of Λ^0, the sum of energy of its daughters is at least the sum of their rest energies, i.e. 0.13957Gev + 0.9383Gev $\approx$ 1.07Gev. In addition, the distribution is also found to be more noisy than that of the strange meson at high energy. This is partially because energetic particles are more likely to pass the filters on transpose momentum and horizontal flight distance. By observing the width of the peak, the uncertainty is estimated to be 0.003 GeV/c^2.

3.2 lifetime of strange meson

3.2.1 the obtained lifetime from data

The plot of number of samples against t with errorbar is shown below. From the general decay pattern, it fits the expectance that the curve is exponential. However, the number entries at 0 and 1 are much smaller than what an exponential model would expect. This will be discussed in subsection 3.2.2.

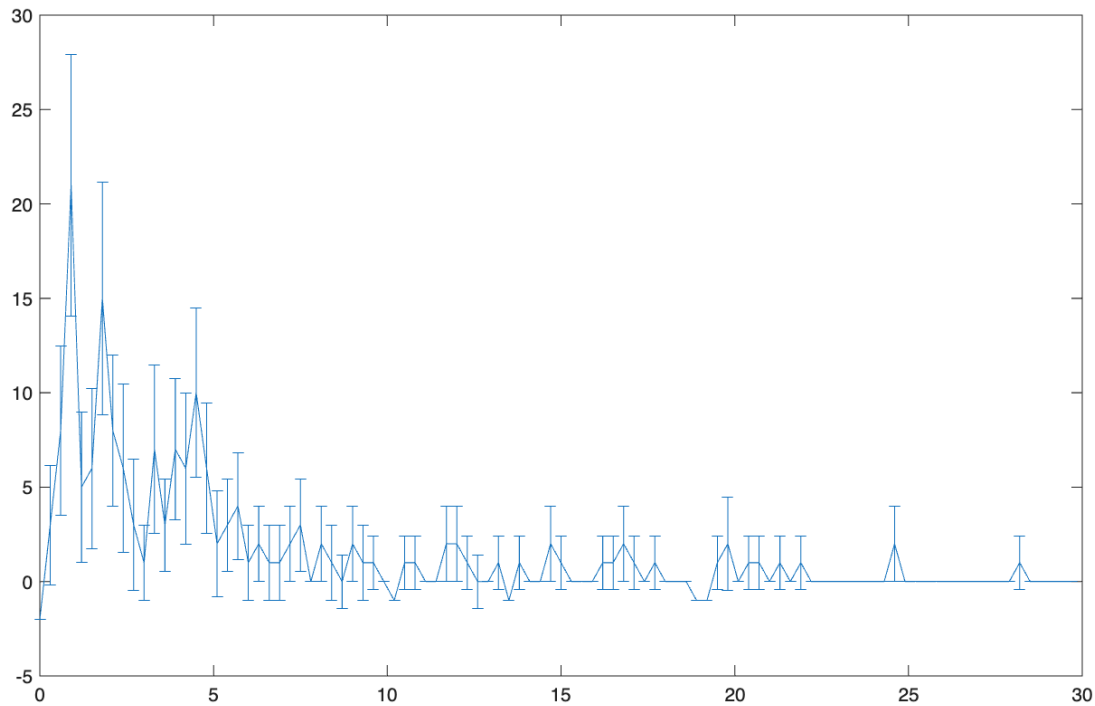

Figure 5: the plot of number of entries against time with errorbar

After qualitatively displaying the shape, the next step is to quantitatively estimate the lifetime. By assuming Poisson error and minimizing the variance, the expected lifetime is found to be $ct = 7.29 \pm 5.14cm$, whose uncertainty is very high. This seems to indicate that an exponential fit with the data is inappropriate. As an additional evidence, the dependence between variance with the normalization constant (A) and standard lifetime (ct) is plotted below. It is seen that the typical expected region of $2cm \leq ct \leq 3cm$ has a relatively high uncertainty.

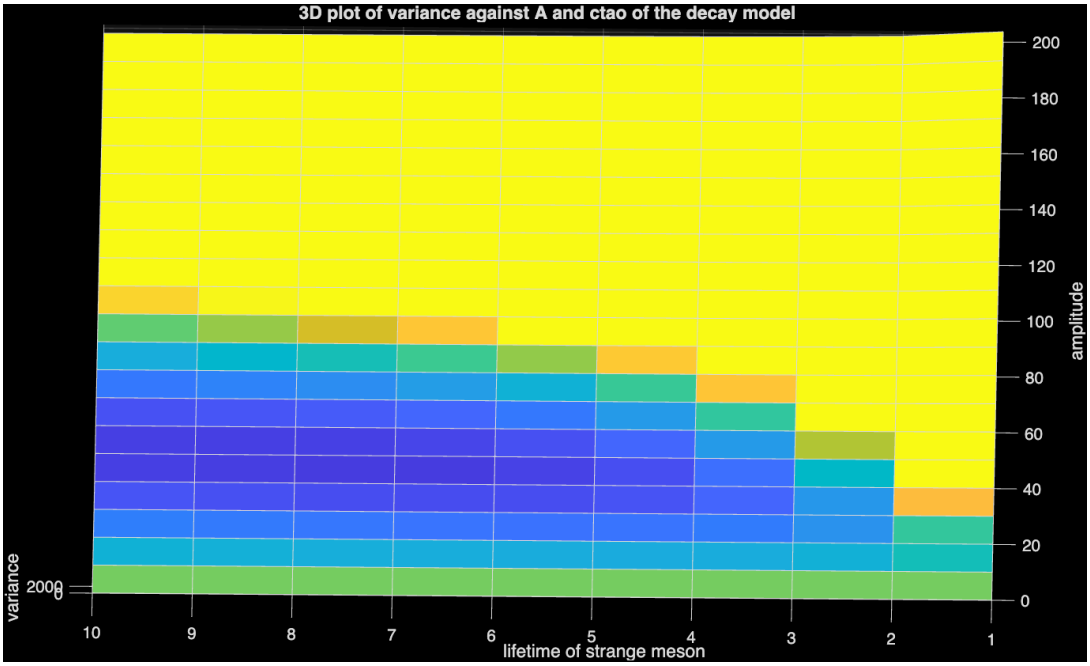

Figure 6: The magnitude of variance viewed from xy plane. The darker colour indicates smaller variance. The base colour represents $\chi^2 \leq 20cm^2$, while the top colour is chosen to represent$\chi^2 \geq 100cm^2$

Usually such a significant error should be caused by mistake in the programme. However, in the process of checking out likely bugs by displaying various features of the lifetime data, it was found that the uncertainty is due to the combination of three major systematic issues. Those errors are discussed in the next section.

3.2.2 likely causes of errors

The three major systematic errors are listed below from the least important to the most important:

- lack of samples. Before removing the filters relevant to lifetime, the mass histogram contains 1423 samples of likely strange mesons. In contrast, the ct and ctside histograms only contain 246 and 70 samples respectively. This small number of candidates increased the random uncertainty.

- Error of errors. The assumption of Poisson errors is reasonable since the expected number of particles with a certain lifetime is a fixed value. However, the error in script is calculated according to the measured data, which contains error by itself.

- Discreteness of histogram. As is found in 5, no entry is found at $ct = 0cm$, while the number of entries at $ct = 1cm$ is much smaller than expected. This justifies the choice of $L_{xy} \geq 2cm$ in section 2.3.1: due to resolution, very few data of strange mesons is accumulated at small distance. The resultant measured lifetime from the inaccurate distribution is thus expected to be larger than the actual one by at least $2cm$.

4 Optimization

With the time complexity of $O(N^2)$, the order of how filters are carried in KOSAnalysis becomes essential. For instance, creation of one new Vertex object via `Vertex(h1,h2)` would be unnecessary if the track did not pass through more economic filters of track such as the cuts on impact parameter and opposite charge.

Most importantly, if a command that can be executed in outer loop is placed in inner loop, the command is going to be executed repetitively for `numel(tracks)` times.

By applying simple optimization, the execution time over the events was successfully shortened to around 16.8 seconds, in contrast with the execution time of over 10 minutes in the least efficient case. (Appendix B.2)

5 Conclusion

The analysis of the given data file has revealed the masses of strange mesons and baryons to be 0.50 ± 0.01 GeV/c^2 and 1.115 ± 0.003 GeV/c^2, which are consistent with the accepted values. Although the result of proper lifetime for strange mesons ($ct = 7.29 \pm 5.14cm$) shows a large uncertainty, it is likely caused by lack of datapoints, invalid assumption of errors and limited resolution of histogram. Overall, the use of a structured modular programming approach proved effective in organising the reconstruction workflow and simplifying debugging and optimisation.

6 Acknowledgements

The data file (cdf.dat) contains experimental data from Collider Detector at Fermilab (CDF), http://www-cdf.fnal.gov, and is used by kind permission of the CDF Collaboration and the Fermi National Accelerator Laboratory (Fermilab), http://www.fnal.gov, for educational purposes. The scripts `CdfDataFile.m`, `CdfEvent.m`, `CdfTrack.m`, `Loop.m`, `Histogram.m` and `Analysis.m` are downloaded from [1]

References

[1] CO03 script. https://www-teaching.physics.ox.ac.uk/CO-MATLAB/html/co03/co03.html, last accessed Feb 22nd 2026.

[2] Kaon. *Wikipedia*, https://en.wikipedia.org/wiki/Kaon, accessed Feb 20th 2026.

[3] Lambda baryon. *Wikipedia*, https://en.wikipedia.org/wiki/Lambda_baryon, accessed Feb 20th 2026.

[4] Detail of Cdf collision. https://www.fnal.gov/pub/tevatron/experiments/cdf.html, accessed Feb 21st 2026

A Appendix A: Essential Code

Listing 1: Source code: CdfDataFile.m

```
classdef CdfDataFile < handle
    % CdfDataFile wraps CDF track data file.
    % This class reads a CDF track data file sequentially.
    % The format:  each event consists of the following:
    %   line 1:  runNumber eventNumber vertex-x vertex-y numTracks
    %     runNumber = CDF run number
    %     eventNumber = CDF event number within run
    %     vertex-x, vertex-y = transverse location (xy) of primary interaction point
    %     numTracks = number of reconstructed tracks
    %   lines 2 to numTracks-2:  cotTheta curvature d0 phi0 z0
    %     cotTheta = polar angle of track at closest approach to (0,0)
    %     curvature = track curvature (actually 0.5*curvature) in xy plane
    %     d0 = impact parameter with respect to origin at point of closest approach
    %     phi0 = azimuthal angle at point of closest approach
    %     z0 = position along beamline at point of closest approach

    properties(Access=protected)
        id; % file identifier
    end

    methods

        % constructor:
        function obj = CdfDataFile(filename)
            obj.id = -1; % invalid
            if ischar(filename)
                % string -> filename
                obj.id = fopen(filename);
            elseif isinteger(filename)
                % integer -> file identifier
                obj.id = filename;
            end
        end

        % return true if file is open (and valid)
        function result = isOpen(obj)
            result = (obj.id > 0);
        end

        % close file
        function close(obj)
            if obj.isOpen()
```

```
                fclose(obj.id);
                obj.id = -1;
            end
        end

        % start reading again from the beginning
        function rewind(obj)
            if obj.isOpen()
                frewind(obj.id);
            end
        end

        % get next event
        % fill next event with the data in datafile
        function event = next(obj)
            event = CdfEvent(); % created in invalid state
            if obj.isOpen()
                while 1
                    str = fgets(obj.id);
                    if str == -1
                        return % end of file - leaves event in invalid state
                    end
                    s = strtrim(str); % trim the leading/ending whitespace
                    if numel(s) <= 0
                        continue % usually just an end of line
                    end
                    if s(1) ~= '#'
                        break; % get on with event
                    else
                        disp(s); % s is obj.id without whitespace
                    end
                end
                % parse event header
                expected = 5;
                [head, n] = sscanf(s, '%d %d %g %g %d', 5);
                %input from string
                % runNumber  eventNumber  vertex-x  vertex-y  numTracks
                %head= vector of successfully read values
                %n=number of successfully read values
                if n == expected
                    % read tracks
                    expected = 5 * head(5);
                    %head(5) is the number of tracks
                    %each track contains five numbers
                    %complete track parameters= CdfTrack properties array
                    %expected number of values from size [5,head(5)]
```

```
                    [pars, n] = fscanf(obj.id, '%g', [5, head(5)]);
                    %input from file
                    %float number, size=[5,head(5)]
                    % reads float values in size
                    %pars is track parameters
                    if n ~= expected
                        return % unexpected end of track data
                    end
                else % unexpected end of header
                    return % leaves event in invalid state
                end
                event.runNumber = head(1);
                event.eventNumber = head(2);
                event.vertex = head(3:4);
                event.tracks = CdfTrack(pars);
                %what is pars?
                %pars is track parameters
            end
        end

    end % methods

end % classdef
```

Listing 2: Source code: CdfEvent.m

```
classdef CdfEvent < handle
    % CdfEvent contains CDF event data

    properties(SetAccess=?CdfDataFile)
        runNumber     % integer:  CDF run number
        eventNumber   % integer:  CDF event number within run
        vertex        % 2x1 float:  transverse location (xy) of primary interaction vertex
        tracks        % array of CdfTrack:  track parameters
    end

    methods

        % constructor:  create CdfEvent in invalid state
        function obj = CdfEvent()
            obj.runNumber = -1;
            obj.eventNumber = -1;
            obj.vertex = [0; 0];
            obj.tracks = [];
        end

        % test if this event is valid
        function valid = isValid(obj)
            valid = (obj.runNumber >= 0);
        end
```

```

    end % methods

end % classdef
```

Listing 3: Source code: CdfTrack.m

```
classdef CdfTrack
    % CdfTrack contains CDF track parameters

    properties(SetAccess=protected)
        cotTheta   % polar angle of track at closest approach to (0,0)
        curvature  % track curvature (actually 0.5*curvature) in xy plane (1/cm)
        d0         % transverse impact parameter at closest approach (cm)
        phi0       % angle in transverse plane at closest approach (radians)
        z0         % position along beamline at closest approach (cm)
    end

    methods

        % constructor:  accepts a 5xN matrix of track parameters
        function obj = CdfTrack(v)
            if nargin > 0
                n = size(v,2); % number of track
                %size(v,1)=number of rows, size(v,2)=number of columns
                obj(n) = CdfTrack; % preallocate array of objects
                %creates CdfTrack objects with size 1*n
                if (size(v,1) == 5)
                    for i = 1:n
                        obj(i).cotTheta  = v(1,i);
                        obj(i).curvature = v(2,i);
                        obj(i).d0        = v(3,i);
                        obj(i).phi0      = v(4,i);
                        obj(i).z0        = v(5,i);
                    end
                else
                    % invalid inputs, so create array of invalid tracks
                    for i = 1:n
                        obj(i) = CdfTrack();
                    end
                end
            else
                % no arguments - create an invalid CdfTrack
                obj.cotTheta = -999.0;
                obj.curvature = -999.0;
```

```
                obj.d0 = -999.0;
                obj.phi0 = -999.0;
                obj.z0 = -999.0;
            end
        end

        % check if this object is valid
        function valid = isValid(obj)
            % originally valid = (obj.cotTheta == -999.0);
            %contains a mistake?
            valid = (obj.cotTheta ~= -999.0);
        end

    end % methods
end % classdef
```

Listing 4: Source code: Loop.m

```
classdef Loop < handle
    %Loop loops over a data file of CDF tracks.

    properties(SetAccess=protected)
        file; % CdfDataFile
    end
    properties % overwrite these public properties
        nevmax; % number of events to process
        report; % number of events between each console report
    end

    methods

        % constructor, using the filename of the data file
        function obj = Loop(filename)
            obj.file = CdfDataFile(filename);
            obj.nevmax = 10000; % (provided cdf.dat has 10000
               events anyway)
            obj.report = 10;
        end

        % close, just in case we need it
        function close(obj)
            obj.file.close();
        end

        % main loop
        function run(obj, analyses)

            % check if there are multiple analyses to run
            if (iscell(analyses))
                ca = analyses;
            else
                ca = { analyses };
```

```
        end

        % check that all the analyses are indeed of the Analysis class
        na = numel(ca);
        for ia = 1:na
            if ~isa(ca{ia}, 'Analysis')
                fprintf(1,'Analysis list contains non-Analysis objects\n');
                return;
            end
        end
        %ca{ia} is an Analysis object from now on:

        % start at the beginning of the data file
        obj.file.rewind();
        %obj.file is the CdfDataFile object

        % call start() on each Analysis
        for ia = 1:na
            ca{ia}.start();
        end
        fprintf(1,'Begin analyses\n');

        % read and analyze events one at a time
        nev = 0; % count events
        while 1
            ev = obj.file.next();
            %CdfEvent object, ev here is a single event
            %this command fill ev with the numbers in file
            if ev.isValid()
                %isValid is a method of CdfEvent
                %=1 when runnumber>=0
                for ia = 1:na
                    ca{ia}.event(ev);
                    %event is a method for Analysis group
                end
                nev = nev + 1;
                if mod(nev, obj.report) == 0
                    fprintf(1,'%d events processed\n', nev);
                end
                if nev >= obj.nevmax
                    break;
                end
            else
                break;
            end
        end

        % call stop() on each Analysis
        for ia = 1:na
```

```
                ca{ia}.stop();
            end

            fprintf(1,'End analysis:  %d events processed\n', nev);
        end

    end

end
```

Listing 5: Source code: helix.m

```
classdef Helix
    % used for geometrical calculations on CdfTrack
    properties(SetAccess=?CdfTrack)
        trk; % a CdfTrack
    end
    properties(Constant)
        kpc = 0.002116;
    end
    methods
    % constructor
        function objarray = Helix(t) %t is an array of CdfTrack objects
        % output an array of helix object with the track property of ith
        % component =t(i)
            if nargin > 0  %number of argument>0
                objarray(numel(t)) = Helix;
                % create an array of Helix objects with the correct size
                for i = 1:numel(t)
                    objarray(i).trk = t(i);
                end
            end
        end

        function r=radius(obj)
            %find radius of circle of track of helix
            r=0.5/obj.trk.curvature;
        end

        function wcxy=center(obj)
        %   find the center of circle at POCA in xy plane
        % wcxy is the 2*1 matrix [wcx,wcy]
        % to be improved
            r=obj.radius();
            d0=obj.trk.d0;
            phi0=obj.trk.phi0;
            wc=(r+d0)*exp(1i*(phi0+pi/2));
            wcxy=[real(wc),imag(wc)];
```

```
            %1i to stand for imaginary number
        end

        function p=points(obj,t)
        % t is the angle made in xy plane
        % between the radius connecting the point of interest
        % and the radius connecting POCA
            r=obj.radius();
            phi0=obj.trk.phi0;
            wcxy=obj.center();
            wc=wcxy(1)+1i*wcxy(2); %complex representation of center
            z0=obj.trk.z0;
            cotTheta=obj.trk.cotTheta;
            q=sign(r); %sign of charge. sign of radius.
            w=wc-1i*r*exp(1i*(phi0+q*t));
            %size(w)=size(t)
            wx=real(w);
            wy=imag(w);
            z=z0+abs(r)*t*cotTheta;
            p=[wx; wy; z];
            % we need a column vector here
            % with multiple inputs (size(t)=1*n)
            %size(p)=3*n, as desired
        end

        function x=intersect(obj,h)
             %h is a helix for which we want to find the intersection with
             %obj
             %x would be a 2-column matrix giving information about
             % two possible intersections
             %first three row contains x,y,z coordinates
             %fourth row contains difference in z at intersection
             %the fifth and sixth row contains t at the intersection
             %viewed from obj and h respectively
             if ~isa(h, 'Helix')
             %isa(object,classname) returns 1 if object is in this class
             %~ means "not"
             %if h is not a Helix object
                 disp('Argument of intersect() is not a Helix');
                 x = [ ]; % no intersection
                 return;
             end

             %whether there is intersection in xy plane:
             c1xy = obj.center();
             c1=c1xy(1)+1i*c1xy(2);
```

```
            c2xy = h.center();
            c2=c2xy(1)+1i*c2xy(2);
            d = c2xy - c1xy; %displacement vector between centers
            d2 = d(1)*d(1) + d(2)*d(2); % square of distance between centers
            r1 = obj.radius();
            r2 = h.radius();
            q1=sign(r1);
            q2=sign(r2);
            if (d2 > (abs(r1)+abs(r2))^2)
                %equavalent to abs(d)>abs(r1)+abs(r2)
                %but we cannot take abs for a vector
                x = [ ]; % no intersection
                return;
            end

            cosAlpha=(d2+r1^2-r2^2)/(2*abs(r1)*sqrt(d2));
            %cos rule
            Alpha=acos(cosAlpha);
            %angle made by the line connecting centers and
            %the line from c1 to intersection
            Delta = atan2(c2xy(2)-c1xy(2), c2xy(1)-c1xy(1));
            %gives correct the quadrant

            wplus=c1+abs(r1)*exp(1i*(Alpha+Delta));
            wminus=c1+abs(r1)*exp(1i*(-Alpha+Delta));
            %complex representations of intersections in xy plane

            phi0obj=obj.trk.phi0;
            phi0h=h.trk.phi0;
            %the phi0 for each of the helix

            %use w=wc-irexp(i(phi0+qt)):
            t1plus=(angle((c1-wplus)/1i/r1)-phi0obj)*q1;
            p1plus=points(obj,t1plus);
            t1minus=(angle((c1-wminus)/1i/r1)-phi0obj)*q1;
            p1minus=points(obj,t1minus);
            t2plus=(angle((c2-wplus)/1i/r2)-phi0h)*q2;
            p2plus=points(h,t2plus);
            t2minus=(angle((c2-wminus)/1i/r2)-phi0h)*q2;
            p2minus=points(h,t2minus);
            %calculate coordinates of both points on both helices

            pplusmean=(p1plus+p2plus)/2;
            pminusmean=(p1minus+p2minus)/2;
            %averaged intersection
            dzplus=abs(p2plus(3)-p1plus(3));
            dzminus=abs(p2minus(3)-p1minus(3));
```

```
            %difference in z

            x=[[pplusmean;dzplus;t1plus;t2plus],[pminusmean;dzminus;t1minus;t2minus]];
            return;
            %t is the actual angle minus angle of POCA (with x axis)
        end

        function x = pt(obj) %momentum magnitude (defined positive)
            %unit is Gev/c
            x = obj.kpc / abs(obj.trk.curvature); %why?
            %by equating centralpetal force with magnetic force
            %(assuming uniform magnetic field in z diretion)
            %one concludes sizept is proportional to radius
        end

        function m=pvector(obj,t) %momentum vector
            %t is the angle made with POCA
            sizept=obj.pt();
            h=obj.trk.curvature;
            q=sign(h);
            cotTheta=obj.trk.cotTheta;
            phi0=obj.trk.phi0;
            %redefine variables for convenience
            px=sizept*cos(phi0+q*t);
            %phi0+q*t is the angle made by momentum vector with x axis
            py=sizept*sin(phi0+q*t);
            pz=sizept*cotTheta;
            m=[px;py;pz];
        end
        %where should this function be located?
        %should it be in vertex or helix class?

        function dpv=ipt(obj,wpvxy) %true impact parameter
            %wpvxy is the transverse location of primary vertex(true interaction)
            %primary vertex is a property of CdfEvent
            r=obj.radius();
            wcxy=obj.center();
            wc=wcxy(1)+1i*wcxy(2);
            wpv=wpvxy(1)+1i*wpvxy(2);
            %wpv is the complex representation
            dpv=abs(wc-wpv)-abs(r);
        end

    end
end % classdef
```

```
            % r=obj.radius;
          %   d0=obj.trk.d0;
```

Listing 6: Source code: vertex.m

```
classdef Vertex
    %note this class is about the intersection of daughter particle helices
    %not to be confused with primary vertex (about where the particle is formed)
    properties(SetAccess=?Helix) %because we need to use functions on helix
        helix; %helices
        loc; %vertex location in xy  2*1
        dz; %difference in z of the two helices (maybe not required)
        t; % angle at the vertex w.r.t both vertices  2*1
    end
    properties(Constant)
        kpc = 0.002116;
    end
    methods
        %contructor
        function obj=Vertex(h1,h2)
            %h1 and h2 are two helix objects
            %returns the position vector with deltaz

            inter=intersect(h1,h2);
            if ~isempty(inter) %check intersection exist
                if inter(4,1)<inter(4,2) %compare dz
                    vertex=inter(:,1); %vertex
                else
                    vertex=inter(:,2);
                end
                obj.helix=[h1,h2];
                obj.loc=vertex(1:3);
                obj.dz=vertex(4);
                obj.t=vertex(5:6);
            else
                obj.helix=[];
                obj.loc=[];
                obj.t=[];
            end
            return;
        end

        function mt=mass(obj,m1,m2)
        %x is the mass of a parent particle that decayed into two tracks.
        %all the masses given are rest mass
        %two mass inputs are required because the identity of decayed
```

```
        %particles is changable
        %all quantities below use natural unit
        h1=obj.helix(1);
        h2=obj.helix(2);
        t1=obj.t(1);
        t2=obj.t(2);
        p1v=h1.pvector(t1);
        p2v=h2.pvector(t2);
        p1=norm(p1v);
        p2=norm(p2v);
        ptv=p1v+p2v; % total momentum vector
        pt=norm(ptv);
        %evaluate magnitude of individual and total momentum

        E1=sqrt(p1^2+m1^2);
        E2=sqrt(p2^2+m2^2);
        Et=E1+E2;
        %calculate energies assuming natural units

        mt=sqrt(Et^2-pt^2);
        end

        function x=Lxyd0pv(obj,wpv)
            %find transverse flight distance Lxy
            %and impact parameter of primary vertex d0pv
            %note this d0pv is not d0 in CdfTrack
            %output x=[Lxy,d0pv]
            h1=obj.helix(1);
            h2=obj.helix(2);
            t1=obj.t(1);
            t2=obj.t(2);
            p1v=h1.pvector(t1);
            p2v=h2.pvector(t2);
            ptv=p1v+p2v; % total momentum vector
            %repetition of code: possibility of improvement?
            pT=ptv(1:2);
            %note ptv(1,2) is the single element in 1st row 2nd column
            pT_unit=pT/norm(pT);
            loc_xy=obj.loc(1:2); %location in xy plane
            deltaloc=loc_xy-wpv;
            Lxy=dot(deltaloc,pT_unit);
            d0pv=deltaloc(1)*pT_unit(2)-pT_unit(1)*deltaloc(2);
            %magnitude of cross product
            x=[Lxy,d0pv];
        end

    end
end
```

Listing 7: Source code: Histogram.m

```
classdef Histogram < handle
    % Histogram accumulates data in pre-defined bins
    % Histogram analyses the statistics of number of datapoints
    % in diffenent intervals (bins)

    properties(SetAccess=protected)
        nbins; % number of bins
        %what is bins?
        %the interval of Histogram. ex. [1,2);[2,3)...
        xlo; % minimum x
        xhi; % maximum x
        data; % bin contents
        underflow; % underflow counter
        overflow; % overflow counter
        %those counts datapoints outside the range (xlo,xhi)
        dx; % bin width
    end

    methods
        % constructor
        function obj = Histogram(nbins, xlo, xhi)
            obj.nbins = nbins;
            obj.xlo = xlo;
            obj.xhi = xhi;
            obj.data = zeros(1,nbins);
            %elements of data records number of datapoints in
               certain bins
            %ex data[1]=30 ->30 datapoints in the first bin
            obj.underflow = 0;
            obj.overflow = 0;
            obj.dx = (xhi - xlo) / nbins;
            %overall range/number of intervals=
            %width of one interval
        end

        % return bin low-edges vector
        function b = bins(obj)
            bb = (obj.xlo : obj.dx : obj.xhi);
            %xlo:interval:xhi,  indeed is low-edges vector
            %ex 1,2 2,3 3,4  -> 1:1:3
            %bb is the interval vector
            %ex  bb[i] is the ith lower-edge of interval
            b = bb(1:obj.nbins);
            %why need this step?
            %elements of b are the indexes of the interval
            %ex  b[i] is the ith interval
        end

        % accumulate data
        function fill(obj, v)
            %what is v?
```

```
            % the function fill v into the histogram?
            for index = 1:numel(v)
                x = v(index);
                if x < obj.xlo
                    obj.underflow = obj.underflow + 1;
                elseif x >= obj.xhi
                    obj.overflow = obj.overflow + 1;
                %counted under/overflow
                else
                    ibin = floor((x-obj.xlo)/obj.dx) + 1;
                    %what is ibin?
                    %index of the interval(bin) which contains x
                    % ex. 1,2 2,3 3,4  x=2.5, floor gives 1, ibin=2
                    obj.data(ibin) = obj.data(ibin) + 1;
                    %number of datapoints stored in the ibinth interval
                    %increments by 1
                end
            end
        end

        % return total number of data points accumulated
        function n = total(obj)
            n = obj.underflow + sum(obj.data) + obj.overflow;
        end

        % return maximum bin contents
        function n = max(obj)
            n = max(obj.data);
            %maximum number of datapoints in one bin
        end

        % plot the histogram
        function plot(obj)
            %plot number of datapoints against index for each bin
            figure;
            binranges = obj.bins();
            %define index vector
            bar(binranges, obj.data, 'histc');
            axis([ obj.xlo obj.xhi 0 obj.max()*1.1 ]);
            %set horizontal and vertical axis limits
        end
    end
end
```

Listing 8: Source code: Analysis.m

```
%to be able to create an array of Analysis,
%and then assign K0SAnalysis and LambdaAnalysis objects to array elements
classdef Analysis < handle & matlab.mixin.Heterogeneous
```

```
    properties

    end
    methods
        function obj = Analysis()
        end
        function start(obj)
        end
        function stop(obj)
        end
        function event(obj)
        end
    end
end
```

Listing 9: Source code: K0SAnalysis.m

```
classdef K0SAnalysis < Analysis
    properties(SetAccess=?Vertex)
        mass;  % Mass histogram

        ct; %lifetime of K0S signal
        ctside; %noise
        lxy;
        lxyside;
    end
    properties(SetAccess=?Helix)
        helix;
    end
    properties(Constant)
        m_pi = 0.13957;  % (charged) Pion mass in Gev/c^2
        m_K0Sworld=0.497614;
    end
    properties(Access=public) %allow changing externally
        minlxy;   % minimum Lxy
        maxd0;    % maximum |d0| allowed
    end

    methods
        %constructor?
        function start(obj)
            % Create mass histogram: 100 bins from 0.4 to 0.6 GeV
               /c^2:
            obj.mass = Histogram(100,0.4,0.6);
            obj.ct = Histogram(100,0,30); % parameter to be
               adjusted
            obj.ctside = Histogram(100,0,30);
            obj.lxy = Histogram(100,0,150); % parameter to be
               adjusted
            obj.lxyside = Histogram(100,0,150);
            if isempty(obj.minlxy) %allow external change
                obj.minlxy = 2.0;   % default require Lxy > 2.0
```

```
                cm
            end
            if isempty(obj.maxd0)
                obj.maxd0  = 0.5;   % default require |d0| < 0.5 cm
            end

        end


        function event(obj,ev)
            % ev is a single event
            if isempty(ev)
                return;
            end
            wpvxy = ev.vertex;   % location of primary vertex
            tracks = ev.tracks;  % get track array
            ntr = numel(tracks); % number of tracks

            % loop over unordered pairs (i<j):
            for i = 1:ntr-1
                %i-related conditions:
                trk1 = tracks(i);
                q1 = sign(trk1.curvature);
                h1 = Helix(trk1);
                pt1 = h1.pt();
                %magnitude of horizontal momentum
                if pt1<= 1.0
                    continue;
                end
                d01=h1.ipt(wpvxy);
                if abs(d01) <= 0.3
                    continue;
                end
                % d01 here is the true impact parameter
                % it is not just the impact parameter of track
                % loop over j for this i:
                for j = i+1:ntr
                    trk2 = tracks(j);
                    q2 = sign(trk2.curvature);
                    if q1 == q2
                        continue;
                    end
                    %  j-related cheap check first: require opposite charge

                    h2 = Helix(trk2);
                    pt2 = h2.pt();

                    % magnitude of transverse momentum of pions (j-related)
```

```
                if pt2<= 1.0
                    continue;
                end

                d02=h2.ipt(wpvxy);
                if abs(d02) <= 0.3
                    continue;
                end
                %requirement on impact parameter of track

                vtx = Vertex(h1, h2);   % primary vertex of helices
                if isempty(vtx.loc)
                    continue;   % no intersection
                end

                Ld = vtx.Lxyd0pv(wpvxy);
                if numel(Ld) < 2
                    continue; %empty Ld for some reasons
                end
                Lxy = Ld(1);      % transverse flight distance
                d0_K0S = Ld(2);   % impact parameter

                % constraint on K0S track
                if Lxy<=obj.minlxy
                    continue;
                end
                if abs(d0_K0S)>=obj.maxd0
                    continue;
                end
                m_K0S = vtx.mass(obj.m_pi, obj.m_pi);
                obj.mass.fill(m_K0S);
                % fill mass analysis histogram


                if m_K0S<0.46||m_K0S>0.54
                    continue;
                end

                t1=vtx.t(1); %the angle arguments
                t2=vtx.t(2);
                pv1=h1.pvector(t1);
                pv2=h2.pvector(t2);
                pvt=pv1+pv2;
                pT=norm(pvt(1:2)); % unit= Gev/c
                %calculate magnitude of horizontal momentum of K0S
                %tao_K0S=Lxy/pT*m_K0S;
                tao_K0S=Lxy/pT*obj.m_K0Sworld;  % unit= cm/c

```

```
                    if m_K0S>=0.48&&m_K0S<=0.52
                        obj.ct.fill(tao_K0S);
                        obj.lxy.fill(Lxy);
                    else
                        obj.ctside.fill(tao_K0S);
                        obj.lxyside.fill(Lxy);
                    end
                    %lifetime histogram
                end % j
            end % i
        end

        function stop(obj)
            % nothing to do here
        end
    end
end
```

Listing 10: Source code: LambdaAnalysis.m

```
classdef LambdaAnalysis < Analysis
    properties(SetAccess=?Vertex)
        mass;  % Mass histogram
    end
    properties(SetAccess=?Helix)
        helix;
    end
    properties(Constant)
        m_pi=0.13957;  % Pion mass in Gev/c^2
        m_pro=0.9383;  % proton mass in Gev/c^2
    end
    methods
        %constructor?
        function start(obj)
            %obj.mass=Histogram(100,1.0,1.2);
            % Create mass histogram: 100 bins from 1.0 to 1.2 GeV
               /c^2
            obj.mass=Histogram(100,1.05,1.3);
        end

        function event(obj,ev)
            if isempty(ev) % avoid bugs caused by inconsistence
               with superclass
                return;
            else
                wpvxy=ev.vertex; % location of primary vertex
                tracks=ev.tracks; % get track array
                ntr=numel(tracks); % number of tracks

                % loop over unordered pairs (i<j)
                for i=1:ntr-1
                    trk1=tracks(i);
```

```
                % Precompute per-i (cheap) quantities once:
                q1=sign(trk1.curvature);
                %sign of charge for the track
                h1=Helix(trk1);
                pt1=h1.pt();
                % transverse momentum of the helix
                d01=abs(h1.ipt(wpvxy));
                % true impact parameter of the track (w.r.t. primary vertex)

                if d01<=0.2
                    continue; % require per-track impact parameter > 0.2 cm
                end

                for j=i+1:ntr % avoid overcounting
                    %similiar structure with i:
                    trk2=tracks(j);
                    q2=sign(trk2.curvature);
                    if q1==q2
                        continue; % require opposite charge
                    end

                    h2=Helix(trk2);
                    d02=abs(h2.ipt(wpvxy));
                    if d02<=0.2
                        continue;
                    end

                    pt2=h2.pt();
                    if max(pt1,pt2)<=2.0
                        continue; % require the higher-pT track to exceed 2 GeV/c
                    end

                    vtx=Vertex(h1,h2); % primary vertex of helices
                    if isempty(vtx.loc)
                        continue; % no intersection
                    end

                    % requirement of lambda
                    Ld=vtx.Lxyd0pv(wpvxy);
                    Lxy=Ld(1); % transverse flight distance
                    d0Lambda=Ld(2); % impact parameter of candidate

                    if Lxy<=2.0
                        continue;
                    end
```

```
                        if abs(d0Lambda)>=0.5
                            continue;
                        end
                        %similiar with K0S

                        if pt1>=pt2
                            m_lambda=vtx.mass(obj.m_pro,obj.m_pi);
                        else
                            m_lambda=vtx.mass(obj.m_pi,obj.m_pro);
                        end
                        % assign masses: proton -> higher-pT track
                        obj.mass.fill(m_lambda); % fill analysis histogram

                    end % j
                end % i
            end %not empty
        end

        function stop(obj)
            % nothing to do here
        end
    end
end
```

Listing 11: Source code: CO03trials.m

```
%%
ka = K0SAnalysis();
ka_lowefficient=K0SAnalysis_lowefficient;
kb=K0SAnalysis();
la = LambdaAnalysis();

kb.minlxy=-999;
kb.maxd0=999;
loop = Loop('cdf.dat');
tic
loop.run(ka_lowefficient); % using the cell array created earlier
toc
%%

%{
figure
%ka.mass.plot();
%kb.mass.plot();
la.mass.plot();
xlabel('mass [GeV/c^2]');
ylabel('entries');
title('distribution of strange baryon mass')
```

```
%title('distribution of strange baryon mass')
%}

bins_array = kb.ct.bins();
%ct_array(1)=[];

ct_array=kb.ct.data;
%ct_array(2)=0;
%ct_array(3)=0;

ctside_array=kb.ctside.data;
A_expected=kb.ct.total()
y_array = kb.ct.data - kb.ctside.data

%y_array(2)=-1;


sigma_array = sqrt(kb.ct.data+kb.ct.data);
%sigma_array = sqrt(kb.ct.data);
%figure
%errorbar(ct_array,y_array,sigma_array);
%plot(ct_array,y_array);
% raw arrays with the same size, will be processed in function


A=0:10:200;
%A=30;
ctao=0:1:10;

%[ax_X,ax_Y]=meshgrid(A,ctao);
% for [X,Y]=meshgrid(x,y) with size(x)=Nx, size(y)=Ny
% size(X)=size(Y)=Ny*Nx

chi2 = chi2K0S(A, ctao, bins_array, y_array, sigma_array); %size=Nx*Ny
mi = min(chi2(:));
[ia_best,ic_best] = find(chi2 == mi);   % lists of all row/col positions with the minimum
A_best=A(ia_best);
ctao_best=ctao(ic_best)
chi2_best=chi2(ia_best,ic_best)

figure
surf(A,ctao,chi2');
caxis([20,100]);
xlabel('amplitude');
ylabel('lifetime of strange meson');
zlabel('variance');
title('3D plot of variance against A and ctao of the decay model')

```

```
% requires transpose

%surf(A,ctao,chi2);

function chi2=chi2K0S(A,ctao,ct_array,y_array,sigma_array)
    % Compute chi^2 for K0S lifetime fit
    % A = amplitude  (trial array) (before meshgrid)
    % ctau=proper lifetime   (trial array)
    % ct_array =summation array of bin centers
    % y_array = signal counts (ct - sideband)  (summation array)
    % sigma_array = uncertainties (summation array)

    filt=(y_array>=0)&(sigma_array~=0); %single |, & are used for array
    %filter out bins with negative counts (unphysical)
    % and no uncertainty (produces infinity)
    ct_array=ct_array(filt);
    y_array=y_array(filt);
    sigma_array=sigma_array(filt);

    %sigma_array(~filt)=1e6;


    % prepare output matrix for possibly vector-valued A and ctao
    nA = numel(A);
    nC = numel(ctao);
    chi2 = zeros(nA,nC);
    sizect=size(ct_array); % for the loop

    % outer loop over A values
    for ia = 1:nA
        A_trial = A(ia);
        % inner loop over ctao values
        for ic = 1:nC
            % create scalar trial values for this loop iteration
            ctao_trial = ctao(ic);

            ctao_array=ctao_trial*ones(sizect);
            A_array=A_trial*ones(sizect);
            %create the two arrays from constant for calculation

            model=A_array./ctao_array.*exp(-ct_array./ctao_array); %model of y_array
            % .operator= operation elements by elements
            % returns an array

            chi2(ia,ic)=sum(((y_array-model).^2)./(sigma_array.^2));
            %chi2(ia,ic)=sum((y_array-model).^2.*(sigma_array.^2));
            %chi2(ia,ic)=sum((y_array-model).^2);
```

```
        end
    end
end
%}
```

B Appendix B: Additional Data

B.1

$$\frac{\int_0^2 exp(\frac{-x}{2.68})dx}{\int_0^\infty exp(\frac{-x}{2.68})dx} \approx 0.53$$

B.2 Efficient code output "Elapsed time is 16.795074 seconds." Least efficient code output 4"Elapsed time is 652.542062 seconds." B.3 When one hat function is convoluted against another hat function of width $2Gev/c^2$, a Guassian distribution will be produced.

C Appendix C: